

PIMID: A Full-System Simulator with Intricacy and Diversity for Processing-in-Memory

Yuan He, Masaaki Kondo, Galen M. Shipman, Jered B. Dominguez-Trujillo, and Kazi Asifuzzaman

Abstract—Processing-in-Memory (PIM) addresses the memory wall by co-locating computation and memory. As real hardware infrastructure remains in early stages, simulation tools are essential for exploring design alternatives and validating architectural concepts before physical implementation. However, existing PIM simulators lack capabilities needed for comprehensive evaluation, including support for diverse memory technologies, flexible processing element placement, networking, host-device co-simulation, and power modeling. We thus introduce PIMID, a full-system simulator providing these missing capabilities for thorough PIM architectural exploration. Built on an extensible design with standardized interfaces, PIMID supports integration of new simulation engines and modeling components to accommodate emerging technologies and evolving requirements.

Index Terms—Processing-in-memory, modeling, simulation, architectural exploration.

I. INTRODUCTION

The growing disparity between processor and memory performance creates significant bottlenecks in modern computing systems, where data movement costs increasingly dominate execution time and energy consumption. PIM architectures offer a promising solution by integrating computation directly within memory arrays, enabling data processing closer to storage and reducing expensive off-chip data transfers [1]. This approach proves particularly beneficial for data-intensive applications including graph analytics, machine learning, and scientific computing where memory bandwidth often limits performance [2].

Despite the potential of PIM architectures, their complexity makes direct hardware evaluation prohibitively expensive and time-consuming, necessitating sophisticated simulation tools. However, current PIM simulation tools face several limitations. Commercial tools like UPMEM SDK [3] focus on specific hardware platforms, while specialized simulators [4]–[6] target particular architectures or use cases. General-purpose academic simulators [7]–[9] attempt broader coverage but typically support only single memory technologies and lack comprehensive host-device co-simulation capabilities necessary

for full-system evaluation. Furthermore, most existing tools provide limited support for fine-grained processing element placement, restricting PEs to fixed locations such as rank or logic die levels, and lack detailed network modeling for intra-memory communication analysis. Power modeling capabilities are often incomplete, focusing only on memory components while neglecting compute elements and network overhead. These constraints limit researchers’ ability to explore the design space and evaluate trade-offs in PIM systems.

To address these limitations, we introduce PIMID, a modular full-system simulator designed to bridge the gaps in current PIM simulation capabilities. PIMID builds upon proven simulation foundations while extending beyond existing tools by supporting heterogeneous memory environments through DRAM modeling via Ramulator [10], SRAM analysis via CACTI [11], and STT-MRAM evaluation via NVSim [12]. PIMID implements true host-device co-simulation through independent simulation domains connected via socket-based communication, maintaining timing causality while allowing independent evolution of host and PIM states. The framework extends MultiPIM’s [9] foundation with fine-grained PE placement from subarrays to logic dies and integrates GARNET [13] for network simulation and McPAT [14] for complete power modeling. Beyond addressing current limitations, PIMID’s extensible design enables integration of new simulation engines and modeling components through standardized interfaces, allowing the framework to evolve with emerging technologies and research requirements.

The contributions of this work include:

- PIMID provides multi-technology memory support enabling direct comparison of DRAM, SRAM, and STT-MRAM under identical workload conditions.
- Host and device co-simulation is implemented through independent simulation domains connected via socket communication.
- Fine-grained PE placement is supported from subarrays to logic dies with both unified and discrete addressing.
- PIMID integrates comprehensive power modeling and advanced network simulation to model intra-memory communication.
- PIMID adopts an extensible design supporting integration of new simulation engines and modeling infrastructure through standardized interfaces.

Manuscript received September X, 2025; revised September XX, 2025.

Y. He is with RIKEN Center for Computational Science, Chuo-ku, Tokyo, Japan. (e-mail: isaacyhe@acm.org).

M. Kondo is with both Keio University, Yokohama, Kanagawa, Japan and RIKEN Center for Computational Science, Kobe, Hyogo, Japan. (e-mail: kondo@acsl.ics.keio.ac.jp).

G.M. Shipman and J.B. Dominguez-Trujillo are with Los Alamos National Laboratory, Los Alamos, NM 87545, USA. (e-mail: {gshipman, jereddt}@lanl.gov).

K. Asifuzzaman is with Oak Ridge National Laboratory, Oak Ridge, TN 37830, USA. (e-mail: asifuzzamank@ornl.gov).

TABLE I: Comparison of PIM Research Tools

Feature	UPMEM SDK [3]	uPiMulator [4]	PiMulator [7]	MultiPIM [9]	PIMID
Simulation Model	Functional	Cycle-accurate	FPGA prototype	Trace-based	Trace-based
Memory Technologies	DRAM	DRAM	DRAM	DRAM	DRAM, SRAM, STT-MRAM
System Scope	PIM device	PIM device	Full-system	Host + PIM	Full-system co-sim
PE Placement	Chip-level	Bank-level	Bank, chip, rank	Rank + logic die	Subarray to logic die
Addressing	Discrete	Discrete	Discrete + unified	Unified	Discrete + unified
VM Support	None	None	DIY	PTW	PTW
Network Model	None	None	DIY	BookSim	GARNET
Power Model	None	None	Measured	Memory only	Host, PE, memory
Extensible	To fill	To fill	To fill	To fill	To fill
Hardware Validation	UPMEM RDIMM	UPMEM RDIMM	FPGA	None	Planned

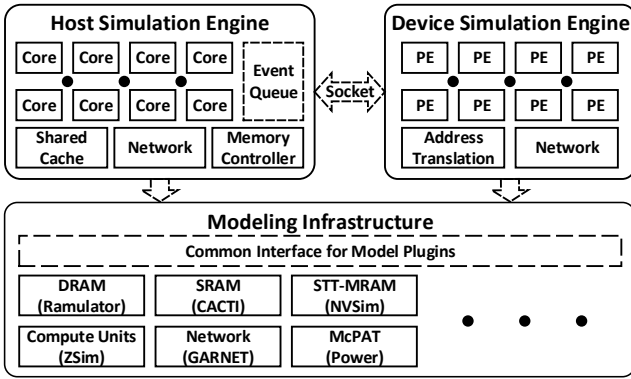


Fig. 1: PIMID system architecture.

II. OVERALL DESIGN OF PIMID: SIMULATION ENGINES AND MODELING INFRASTRUCTURE

PIMID implements a domain-based architecture consisting of two simulation engines and a shared modeling infrastructure, as illustrated in Figure 1. The two engines operate independently but coordinate through socket-based communication and share access to a unified modeling infrastructure. This approach addresses key limitations in existing PIM simulation tools, as summarized in Table I.

PIMID’s domain-based architecture distinguishes it from existing tools through true host-device co-simulation, multi-technology memory support, and comprehensive modeling infrastructure. Configurations of the simulation engines are specified in YAML, enabling flexible system specification and easy modification of simulation parameters. When executing workloads with PIMID, annotated code regions are offloaded to device simulation engines while the remaining code executes on the host engine.

Host Simulation Engine. The host simulation engine handles conventional processor simulation including instruction execution, cache hierarchy management, and system-level operations. Unlike MultiPIM’s single-process approach, PIMID runs the host engine as an independent ZSim instance with its own timing state and performance counters. The host engine processes application code until encountering device offload

annotations, at which point it initiates communication with the PIM simulation engine through the socket interface. This separation enables realistic modeling of host-device interaction latencies and allows independent optimization of host and device simulation parameters.

Device Simulation Engine. The device simulation engine simulates memory-side PEs and their local memory hierarchies. Similar to the host engine, the device engine operates as an independent ZSim instance but configured as PEs for memory-bound operations. The engine receives offloaded tasks from the host engine and executes them using memory-side PEs. Task scheduling and resource allocation within the PIM domain are handled by a dedicated extensible scheduler that maps operations to specific PEs based on their placement within the memory hierarchy.

Modeling Infrastructure. The shared modeling infrastructure provides simulation models accessible by both simulation engines. This infrastructure incorporates multiple memory simulators: Ramulator [10] for DRAM timing and power modeling, CACTI [11] for SRAM analysis, and NVSim [12] for emerging non-volatile memories like STT-MRAM, extending beyond MultiPIM’s DRAM-only support. Network modeling uses GARNET [13] for cycle-accurate communication between PEs and memory arrays, replacing MultiPIM’s BookSim integration with more detailed topology and flow control modeling. McPAT [14] integration provides comprehensive power estimation across both simulation engines, including PEs, memory arrays, and network components, addressing MultiPIM’s limitation of memory-only power modeling.

Extensible Design. PIMID supports seamless integration of new simulation engines and modeling components through well-defined interfaces. Standardized APIs provide timing synchronization, data exchange, and performance reporting capabilities, with all interfaces defined in YAML format to enable users to replace existing engines or add new modeling infrastructure without core modifications. This plugin-based design allows users to extend PIMID’s capabilities.

III. KEY FEATURES AND IMPLEMENTATION DETAILS

PIMID addresses critical gaps in existing PIM simulation tools through several key innovations. This section describes these features and their technical implementation details that enable comprehensive architectural exploration.

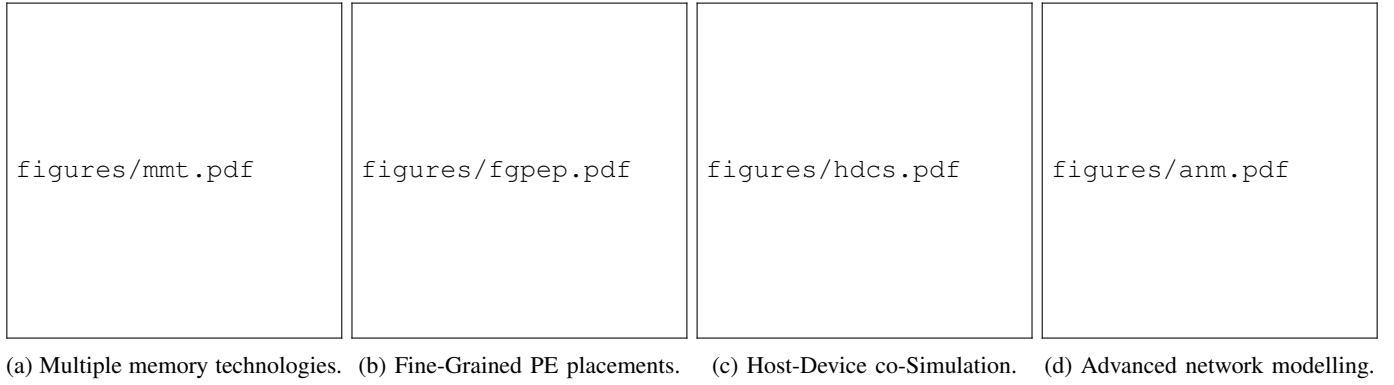


Fig. 2: Implementation details of key features provided by PIMID.

A. Support for Multiple Memory Technologies

By integrating multiple memory models, such as Ramulator [10], CACTI [11], and NVSim [12], PIMID has enabled simulations with DRAM, SRAM and STT-MRAM. The implementation uses a memory abstraction layer that standardizes interfaces for different memory models and handles technology-specific parameters. When a specific memory model is picked, memory requests are then routed to it for timing and energy evaluations.

B. Fine-Grained PE Placement

PEs can be placed within subarrays, banks, chips, ranks, or logic layers, providing granularity that significantly exceeds existing simulators. The implementation maintains hierarchical placement tables that associate PEs with specific memory hierarchy levels and their corresponding addressing modes.

For unified addressing at rank level, the system extends directory-based coherence protocols to track data sharing between host and PIM domains. For discrete addressing at finer granularities, the implementation provides explicit address translation tables for each PE and assumes explicit data communication between PEs and their corresponding memory devices.

C. Host-Device Co-Simulation

Independent simulation engines represent host and PIM domains, synchronized through socket-based communication that maintains timing causality across domains. The implementation uses sockets with a lightweight protocol to handle event synchronization, timing coordination, and data exchange between the host and device. A centralized event queue maintains global timing order across simulation domains.

D. Comprehensive Power and Energy Modeling

PIMID integrates McPAT [14] for system-wide energy estimation. The implementation combines McPAT output with memory-specific power models through a unified energy accounting system that tracks consumption at component granularity.

E. Advanced Network Modeling

GARNET [13] provides flexible cycle-accurate network models with configurable topologies, flow control mechanisms, and routing algorithms. The implementation maps memory and PE at different hierarchy levels as network nodes. With GARNET, impacts from packet routing, flow control, and congestion can be accounted to system performance.

F. Extensible Framework Architecture

PIMID implements a plugin-based design supporting integration of new simulation engines and modeling infrastructure through standardized interfaces. The implementation includes a component registration system where new components register through YAML-based interface specifications that define protocols for data exchange and performance counting.

Plugin inclusion occurs at startup through configuration file parsing, with the runtime coordinator instantiating registered components and establishing communication channels. The implementation supports both component replacement and capability addition without requiring core framework modifications.

G. Programming Model Support

PIMID inherits MultiPIM's approach by marking code regions for PIM execution through annotations. Multi-thread Workloads can be compiled with OpenMP or MPICH to mimic shared memory and message passing programming models in the simulation. The implementation preserves shared memory semantics for OpenMP regions executing on PEs through hardware coherence mechanisms, while inter-PE communication uses explicit message passing handled based on data access patterns and PE placement.

IV. EVALUATION AND DISCUSSION

PIMID's evaluation framework provides comprehensive system characterization through performance analysis, energy modeling, and scalability assessment. The simulator tracks system throughput, memory access latency, network utilization, and execution time breakdown across host and PIM domains, with detailed component-level statistics enabling bottleneck identification and optimization guidance.

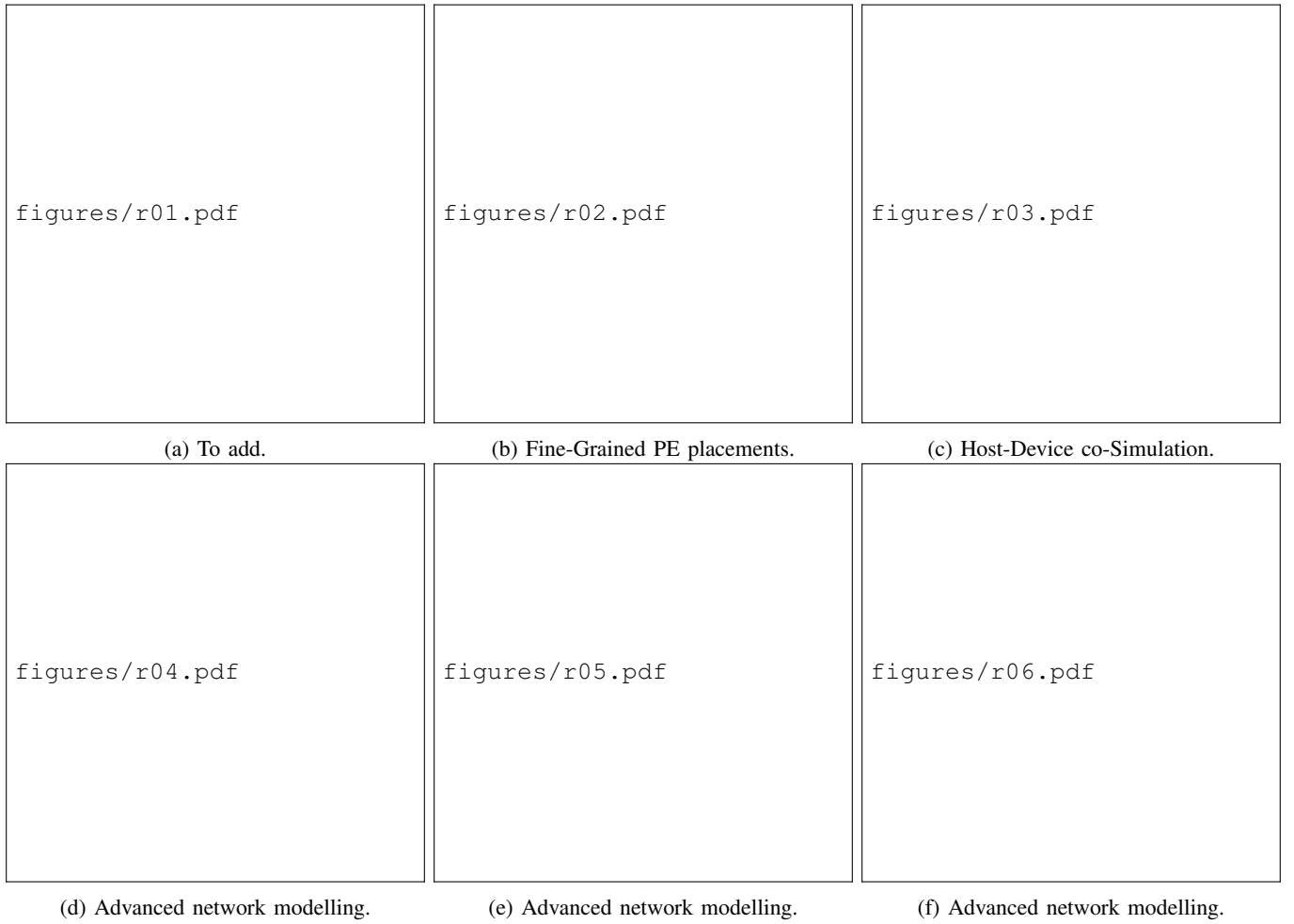


Fig. 3: Sample Evaluation Results with PIMID.

Scalability evaluation examines system behavior across varying PE counts, memory stack configurations, and network topologies, supporting configurations from small-scale systems to large-scale deployments with hundreds of processing elements distributed across multiple memory stacks.

A. Performance Validation through Sample Simulations

To write...

B. Current Limitations and Future Development

One of the limitations PIMID currently faces is limited processing element models that may not capture the full range of emerging PIM architectures. The system provides comprehensive modeling for conventional processing elements but may require extensions for specialized compute units with unique instruction sets or execution patterns. Current PE models assume general-purpose in-order or out-of-order CPU cores adapted for memory-side computation, but emerging PIM designs may incorporate wider options such as neuromorphic processors, vector accelerators, or domain-specific compute units that require different modeling approaches. This limitation becomes more significant as PIM architectures evolve

toward heterogeneous processing elements with specialized capabilities.

Another important feature missing is hardware validation, which could be complemented with FPGA prototyping to verify simulation accuracy against real implementations. Without hardware validation, users must rely on simulation-only results, which may not fully capture the complexities of actual PIM systems including manufacturing variations, thermal effects, and real-world operating conditions. FPGA-based validation would provide confidence in simulation results and help identify discrepancies between modeled and actual system behavior.

Future developments will address these limitations through expanded PE modeling frameworks that support diverse architectural configurations and FPGA-based validation platforms.

V. CONCLUSIONS

In this paper, we introduce PIMID, a full-system simulator that addresses critical gaps in PIM simulation infrastructure through true host-device co-simulation, multi-technology memory support, and fine-grained processing element placement. PIMID provides an extensible framework that integrates established simulation tools (Ramulator, CACTI, NVSim, GARNET, McPAT) with novel co-simulation capabilities to

enable comprehensive architectural exploration. The simulator's modular design supports diverse memory technologies, flexible PE placement from subarrays to logic dies, and realistic power and network modeling not available in existing tools. We expect that future studies can better identify design trade-offs and validate PIM architectures through more precise simulation of complex memory-centric computing systems using PIMID.

REFERENCES

- [1] O. Mutlu *et al.*, "Processing data where it makes sense: Enabling in-memory computation," *Microprocessors and Microsystems*, vol. 67, pp. 28–41, June 2019.
- [2] —, "A modern primer on processing in memory," *arXiv preprint arXiv:2012.03112v5*, February 2025.
- [3] UPMEM, *UPMEM SDK*, available at <https://sdk.upmem.com>.
- [4] B. Hyun *et al.*, "Pathfinding Future PIM Architectures by Demystifying a Commercial PIM Technology," in *Proc. of HPCA'24*, March 2024, pp. 263–279.
- [5] D. Christ, L. Steiner, M. Jung, and N. Wehn, "PIMSys: A Virtual Prototype for Processing in Memory," in *Proc. International Symposium on Memory Systems*, 2024, pp. 26–33.
- [6] Y. Shin, J. Park, S. Cho, and H. Sung, "PIMCoSim: Hardware/Software Co-Simulator for Exploring Processing-in-Memory Architectures," *Electronics*, vol. 13, no. 23, 2024.
- [7] S. Mosanu *et al.*, "PiMulator: a Fast and Flexible Processing-in-Memory Emulation Platform," in *Proc. of DATE'22*, March 2022, pp. 1473–1478.
- [8] B. E. Forlin *et al.*, "Sim²PIM: A Complete Simulation Framework for Processing-in-Memory," *Journal of Parallel and Distributed Computing*, 2022.
- [9] C. Yu *et al.*, "MultiPIM: A Detailed and Configurable Multi-Stack Processing-In-Memory Simulator," *IEEE CAL*, vol. 20, no. 1, pp. 54–57, 2021.
- [10] Y. Kim *et al.*, "Ramulator: A Fast and Extensible DRAM Simulator," *IEEE CAL*, vol. 15, no. 1, pp. 45–49, 2016.
- [11] N. Muralimanohar *et al.*, "Optimizing NUCA Organizations and Wiring Alternatives for Large Caches with CACTI 6.0," in *Proc. of 40th MICRO*, December 2007, pp. 3–14.
- [12] X. Dong *et al.*, "NVSim: A Circuit-Level Performance, Energy, and Area Model for Emerging Nonvolatile Memory," *IEEE TCAD*, vol. 31, no. 7, pp. 994–1007, 2012.
- [13] T. Krishna, *Garnet2.0: A Detailed On-Chip Network Model Inside a Full-System Simulator*, available at https://bpb-us-e1.wpmucdn.com/sites.gatech.edu/dist/c/332/files/2017/09/Garnet2.0-Tutorial_gem5-workshop_ARM-Rsh-Summit2017.pdf.
- [14] S. Li *et al.*, "McPAT: An Integrated Power, Area, and Timing Modeling Framework for Multicore and Manycore Architectures," in *Proc. of 42nd MICRO*, December 2009, pp. 469–480.